

New York City · Paris · Berlin

Predicting Airbnb Listing Price Across Different Cities

UE24CS352A Machine Learning · Mini Project 71

Dhanush S S (PES1UG24AM082)
Harsha K R (PES1UG24AM114)

Hosts guess their price; we learn it from data

1

Predict

the nightly price of a listing from its size, room type, location, amenities, reviews, host details and description

2

Compare

five models: linear regression, KNN, random forest, XGBoost and a neural network

3

Transfer

test whether one model can serve several cities, and price listings in a city it has never seen

Why it matters

- New hosts have no pricing system, unlike hotels
- Guests want to know whether a price is fair
- Rules differ by city (e.g. NYC's 2023 short-term rental law), so a model that travels is harder than it looks

75,222 listings from Inside Airbnb, June/July 2026

New York City

20,890

listings · median $170/night · 78% need 28+ nights

Paris

46,000

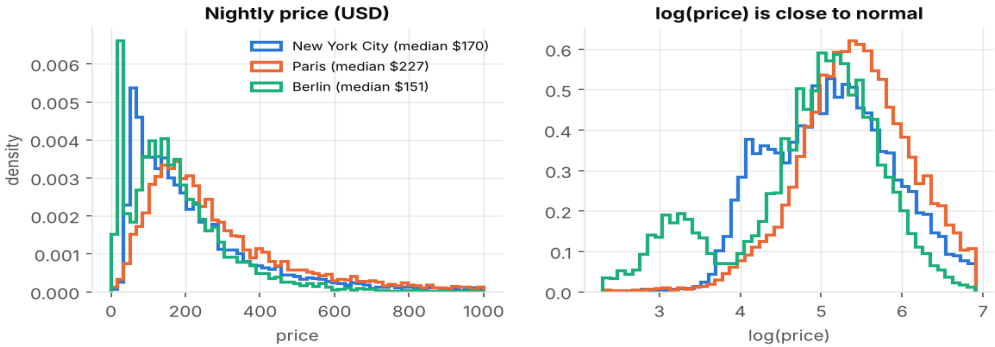
listings · median $227/night · 14% need 28+ nights

Berlin

8,332

listings · median $151/night · 20% need 28+ nights

- 90 columns per listing: size, room and property type, location, amenities, reviews, host details, title and description
- Kept listings priced 10 to 1000 USD per night; EUR converted to USD at a fixed 1.15
- Fixed 80/20 train/test split per city, used by every experiment
- Target is log(price): raw prices are heavily skewed, logs are close to normal



From raw listings to a price in five steps

1

Clean

parse prices, bathrooms, amenities; drop missing or extreme prices

2

Engineer

distance to centre, review ages, shared-bath flag, licence flag

3

Encode

one sklearn ColumnTransformer: scale, one-hot, multi-hot, TF-IDF + SVD

4

Train

five regressors on log(price); XGBoost tuned by 3-fold random search

5

Evaluate

R^2 and MSE on log price, MAE in dollars, on held-out test sets

Leakage control

Columns computed from the price (price_quote_*, estimated_revenue_l365d) and the length of the quoted stay are never used. The pipeline is fitted on training data only, so the test set never influences scaling, TF-IDF vocabulary or SVD.

189 features in three groups

C Continuous

55 columns

guests, bedrooms, beds, bathrooms, minimum nights, availability, reviews and 7 review scores, host tenure, superhost, licence, distance to centre, lat/lon (+ missing-value flags)

O Categorical

84 columns

room type and property type (one-hot) plus the 60 most common amenities (multi-hot: wifi, kitchen, dishwasher, air conditioning …)

T Text

50 columns

title + description → TF-IDF on unigrams and bigrams → truncated SVD to 50 dimensions (same idea as the reference report)

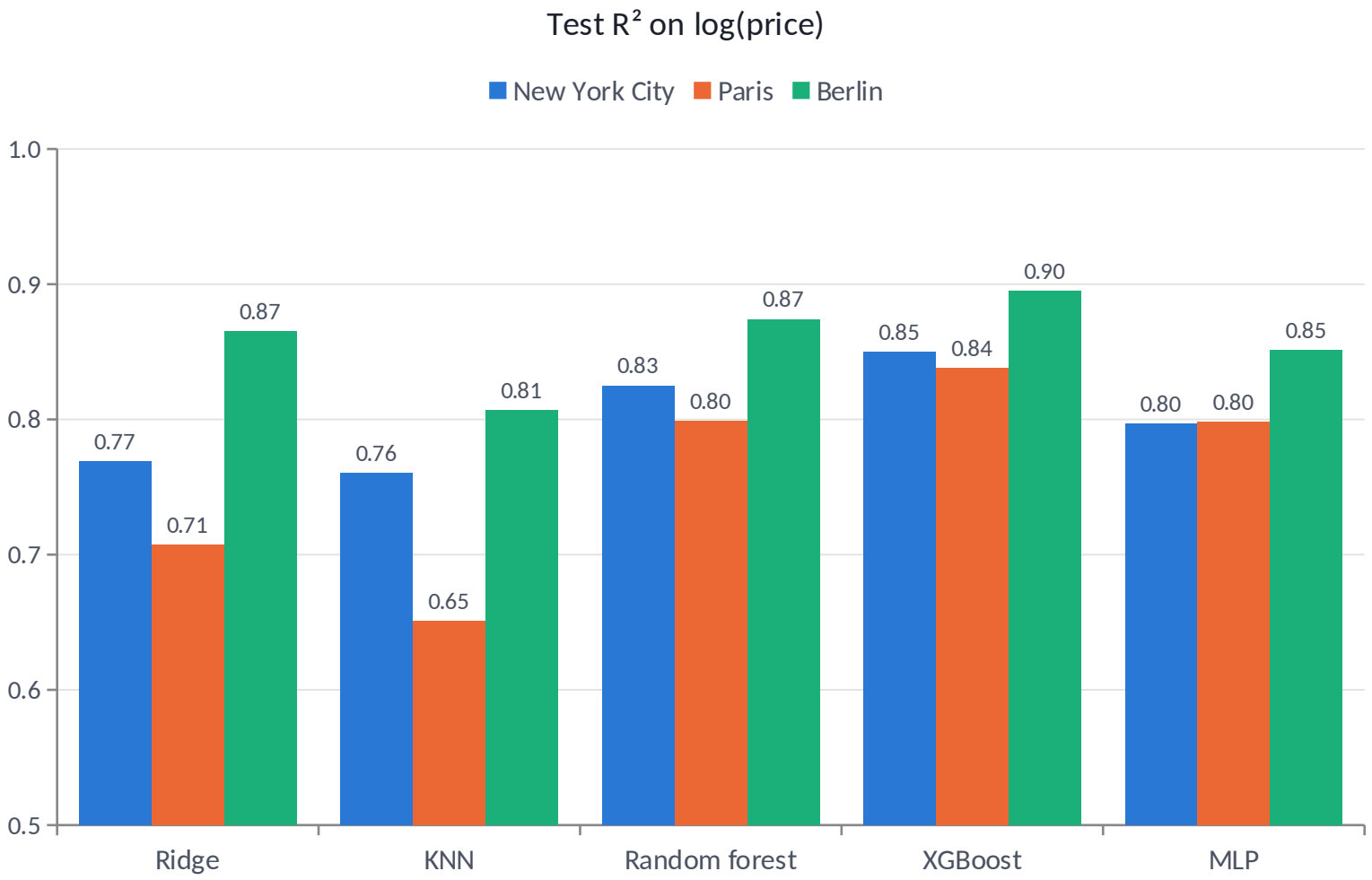
The final multi-city model also gets a one-hot city indicator. The transfer experiments drop lat/lon and keep distance to centre, which means the same thing in every city.

Five models, from baseline to boosted trees

Ridge regression	Baseline. Linear in the features, L2 penalty (α = 10).
K-nearest neighbours	Baseline. Average price of the 15 most similar listings, distance-weighted.
Random forest	300 trees on bootstrap samples, a third of features per split, averaged.
XGBoost	1,170 boosted trees, depth 8, learning rate 0.0225, row and column subsampling; tuned by 3-fold random search on NYC (CV R^2 0.848).
Neural network (MLP)	Two hidden layers (64, 32, ReLU), Adam, L2 = 0.001, early stopping on 10% validation.

All models predict log(price) and are scored on the same held-out listings.

XGBoost wins in every city



0.85

NYC test R^2

0.84

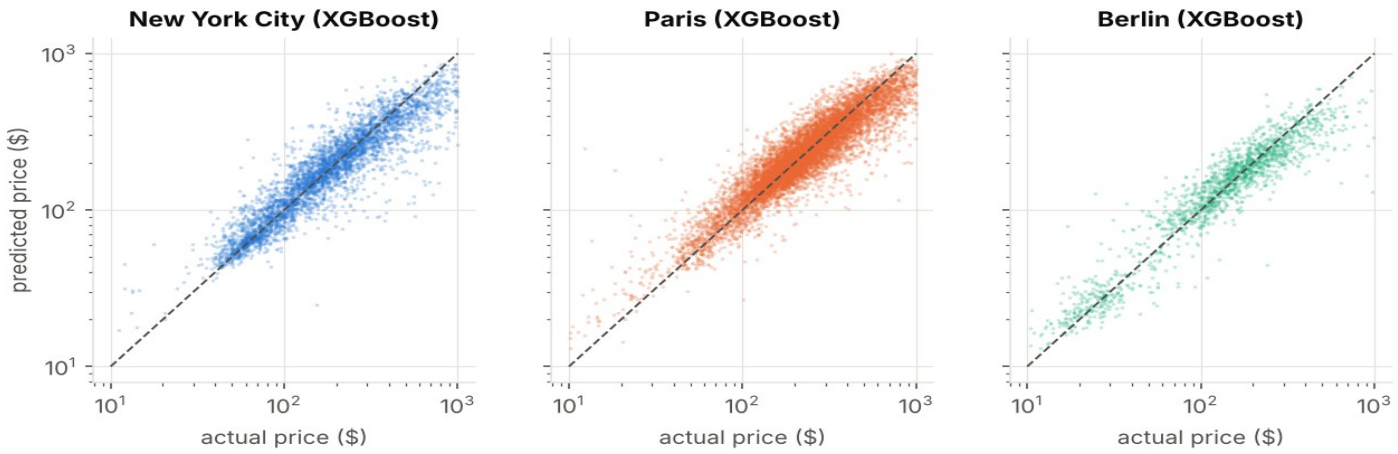
Paris test R^2

0.90

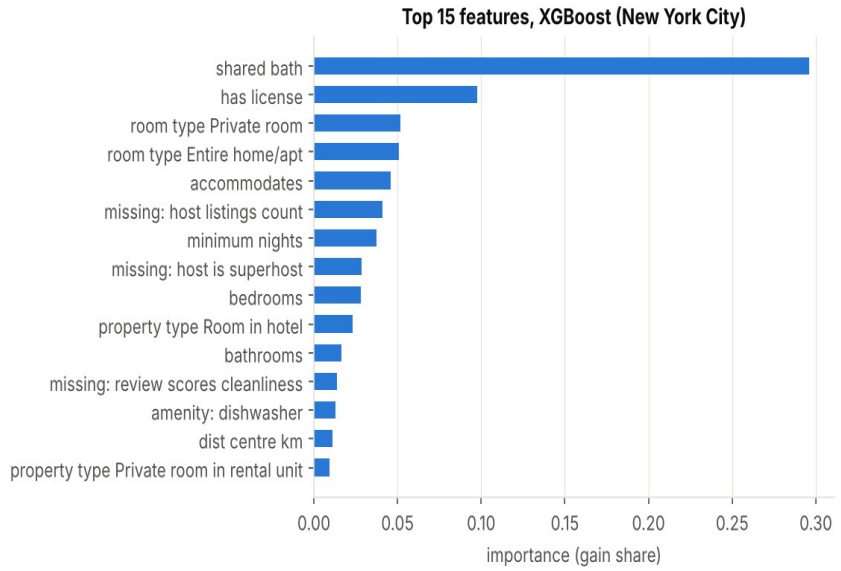
Berlin test R^2

Typical error ≈ 15% of the price (MAE $50 NYC, $57 Paris, $39 Berlin). Reference report: 0.74 NYC, 0.70 Paris.

Predictions track real prices across the range



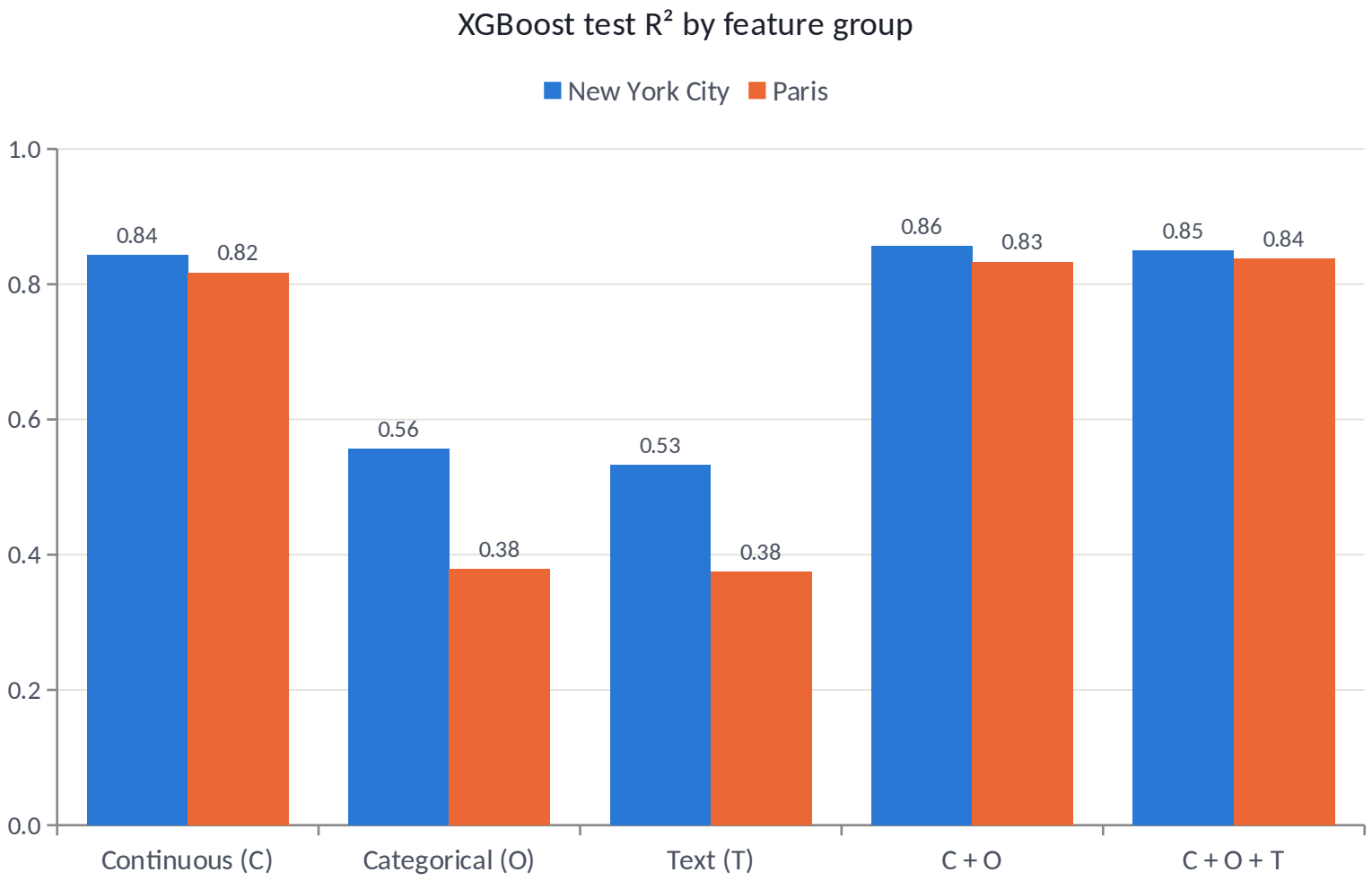
Each dot is a held-out listing; the dashed line is a perfect prediction.



What drives price

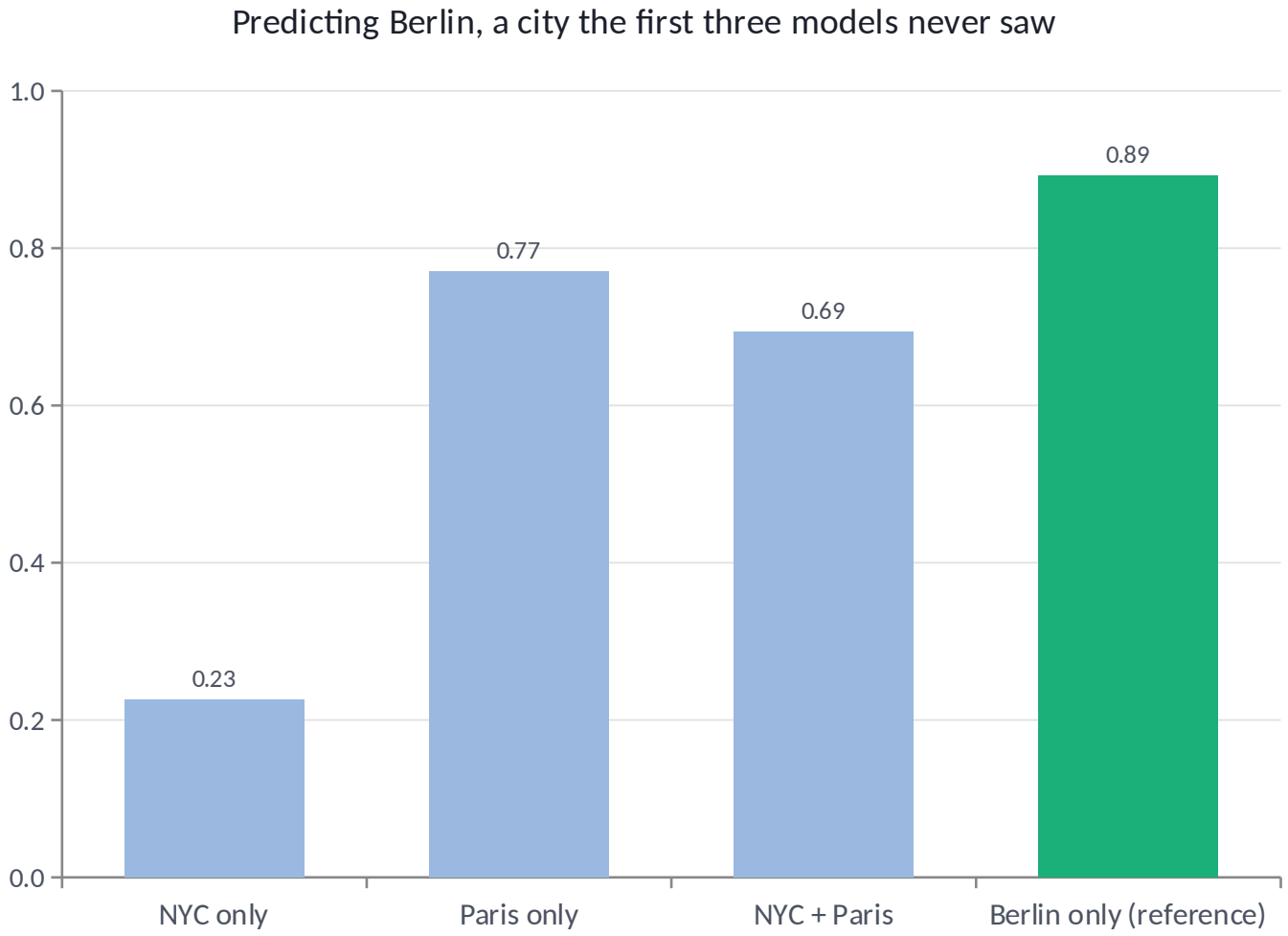
- Shared vs private bathroom and room type: a room in a shared flat is far cheaper
- Licence status and minimum stay: long-stay listings are quoted at monthly-discount rates
- Capacity (guests, bedrooms, bathrooms), then location (distance to centre) and amenities

Structured features carry the signal; text adds little



- Continuous features alone reach 0.84 / 0.82
- Categorical + amenities add ≈ 0.015
- Text alone is weak (0.53 / 0.38) and adds almost nothing on top: descriptions mostly restate size and location

A similar market transfers; more data does not



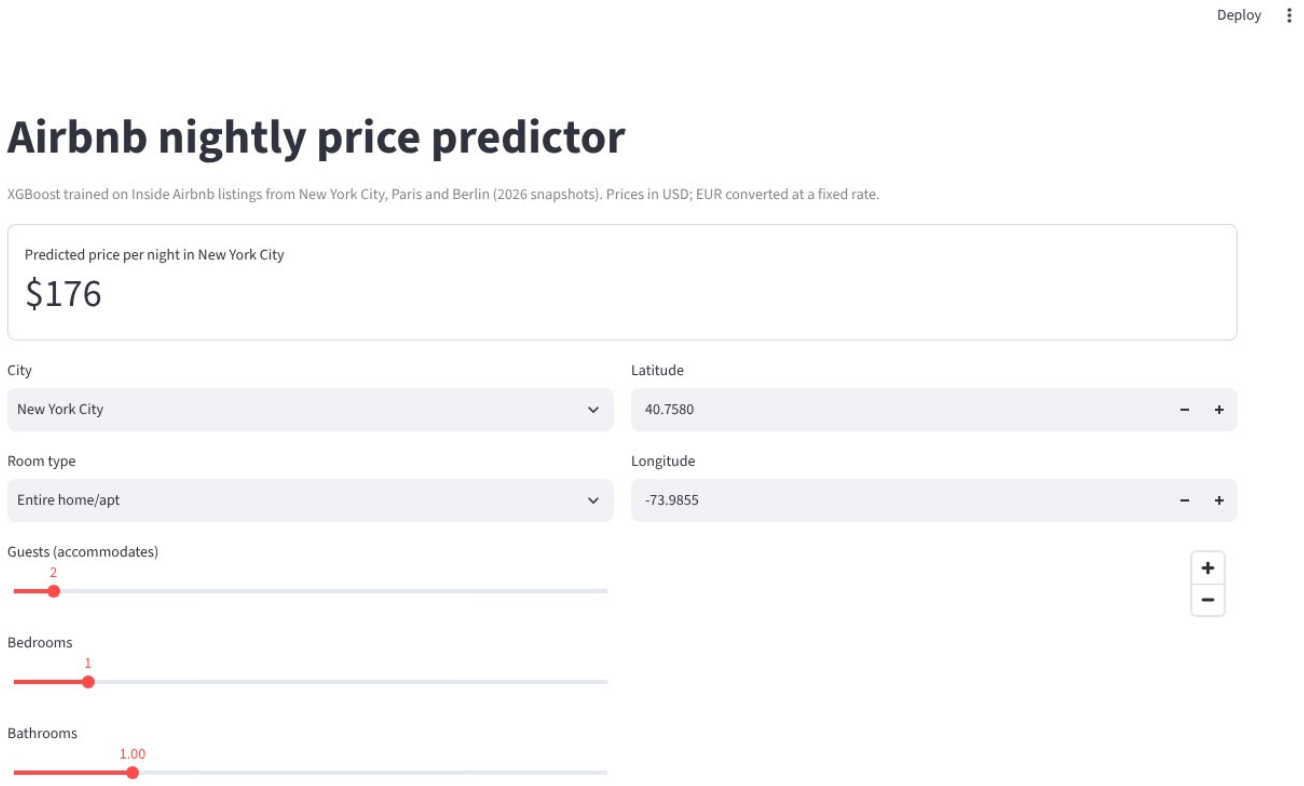
Combined training

One model on NYC + Paris scores 0.836 / 0.825, almost the same as two separate models (0.844 / 0.829).

Why NYC fails

Since NYC's 2023 short-term rental law, 78% of its listings need 28+ night stays (Paris 14%, Berlin 20%). NYC prices follow different rules, so its model misleads in Europe.

Live demo: price any listing in three cities




```
streamlit run app/app.py
```

Demo script

1. Pick a city and room type; change guests and bedrooms; watch the price move
2. Switch NYC → Paris → Berlin with the same flat
3. Set minimum nights to 30 and see the monthly-rate drop
4. Terminal: python scripts/predict.py --sample 5 shows real test listings, actual vs predicted

Conclusions

- XGBoost on log(price) predicts within ≈15%: R^2 0.85 NYC, 0.84 Paris, 0.90 Berlin
- Size, room type, bathroom sharing, licence and minimum stay matter most; text adds little
- One model can serve several cities; transfer to a new city works from a similar market (Paris → Berlin 0.77), not from NYC
- Next: seasonality and calendar data, booked prices, domain adaptation for new cities

Thank you · Questions?

Dhanush S S (PES1UG24AM082) · Harsha K R (PES1UG24AM114)

Code, data and results: GitHub repository (see README)